

Inteligencia Artificial y Simulación

Proyecto Final — Curso 2025–2026

rl-competitive-training-scheduler

Construcción de rutas de aprendizaje mediante
Deep Q-Network con evaluación pedagógica por LLM

Glenda N. Rios Rodríguez

Grupo: C312

11 de junio de 2026

Índice

1. Descripción del Problema	2
2. Modelado Formal	2
2.1. Proceso de Decisión de Markov	2
2.2. Representación de la acción para la red Q	5
3. Descripción del Dataset	5
3.1. Fuentes de datos	5
3.2. Pipeline de preprocesamiento	6
3.3. Perfiles de estudiante	7
4. Diseño del Algoritmo	7
4.1. Deep Q-Network (DQN)	7
4.2. Algoritmos de referencia (baselines)	8
5. Rol del LLM en el Sistema	9
5.1. Rol 1: Generación de perfiles de estudiante realistas	9
5.2. Rol 2: Evaluación pedagógica de sesiones completadas	10
6. Metodología Experimental	11
6.1. Entrenamiento del agente DQN	11
6.2. Protocolo de evaluación	11
7. Resultados y Análisis	12
7.1. Análisis de la recompensa total	12
7.2. Progresión de dificultad (Spearman)	12
7.3. Cobertura temática	13
7.4. Tasa de éxito	13
7.5. Mejora de rating	13
7.6. Análisis cualitativo por tipos de perfil	13
7.7. Síntesis de los resultados	14
8. Limitaciones y Posibles Mejoras	14
8.1. Limitaciones actuales	14
8.2. Posibles mejoras	15

1. Descripción del Problema

En la programación competitiva, la calidad del entrenamiento no depende solo de la cantidad de problemas resueltos, sino de *qué* problemas se intentan y en *qué orden*. Un estudiante con 120 minutos disponibles puede malgastar su sesión en problemas triviales, repetir temáticas ya dominadas, o enfrentar problemas demasiado difíciles que generan frustración sin aprendizaje real.

El problema que aborda este proyecto es el siguiente:

Dado un conjunto de N problemas de programación competitiva y un presupuesto de tiempo T minutos, seleccionar y ordenar una subsecuencia de problemas que maximice el aprendizaje del estudiante, respetando las restricciones temporales y adaptándose al estado cognitivo actualizado después de cada intento.

Este problema presenta tres características que lo hacen especialmente difícil de resolver con métodos clásicos:

- **Secuencialidad:** cada decisión modifica el estado del estudiante (ELO por tema, fatiga, tiempo restante), lo que hace que las decisiones futuras dependan de las actuales.
- **Incertidumbre:** el resultado de cada intento (resuelto/fallido, tiempo empleado) es estocástico y depende del estado interno del estudiante.
- **Objetivos múltiples:** una buena sesión debe equilibrar tasa de éxito, cobertura temática, progresión de dificultad y eficiencia temporal.

El proyecto enmarca este problema como un **Proceso de Decisión de Markov (MDP)** y lo resuelve con un agente de aprendizaje por refuerzo basado en Deep Q-Network (DQN), al que se añade un módulo de evaluación pedagógica mediante un modelo de lenguaje grande (LLM) local.

2. Modelado Formal

2.1. Proceso de Decisión de Markov

El sistema se modela como un MDP finito por episodio con la tupla $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma \rangle$, donde:

Espacio de estados $\mathcal{S}$. El estado $s_t \in \mathbb{R}^{24}$ en el paso t codifica el estado completo del estudiante:

$$s_t = [r_{\text{norm}}, f_t, t_{\text{used,norm}}, sr_t, \text{ELO}_1^{\text{norm}}, \dots, \text{ELO}_{20}^{\text{norm}}]$$

donde r_{norm} es el rating global normalizado, $f_t \in [0, 1]$ la fatiga acumulada, $t_{\text{used,norm}}$ el tiempo consumido relativo al presupuesto, sr_t la tasa de éxito hasta el momento, y $\text{ELO}_k^{\text{norm}}$ el ELO normalizado para cada uno de los 20 temas canónicos.

Espacio de acciones $\mathcal{A}$. La acción $a_t \in \{0, 1, \dots, N-1\}$ es el índice del problema a intentar en el paso t , sobre un pool fijo de $N = 500$ problemas. Las acciones inválidas (problemas ya intentados o cuyo tiempo estimado supera el tiempo restante) se enmascaran con $-\infty$ en el cómputo de Q-valores, garantizando que el agente solo seleccione acciones factibles.

Función de transición $\mathcal{T}$. La transición $\mathcal{T}(s_{t+1} | s_t, a_t)$ está definida implícitamente por el modelo del estudiante. La probabilidad de resolución del problema p dado el estado del estudiante sigue un modelo ELO logístico:

$$P_{\text{solve}}(p | s_t) = \sigma\left(\frac{r_{\text{eff}}(s_t) - r_p}{\theta}\right)$$

donde σ es la función sigmoide, r_p es el rating del problema, $r_{\text{eff}}(s_t) = \min_k \text{ELO}_k$ sobre los temas del problema (hipótesis del eslabón más débil), y $\theta = 400$ es el parámetro de escala ELO estándar.

Si el intento es exitoso, los ELO por tema se actualizan:

$$\Delta \text{ELO}_k = C_{\text{elo}} \cdot (1 - P_k) \cdot \text{challenge}_k \cdot (1 - f_t), \quad |\Delta \text{ELO}_k| \leq 30$$

La fatiga se incrementa proporcionalmente al tiempo empleado, con $\lambda_{\text{fatigue}} = 1, 2$.

Función de recompensa $\mathcal{R}$. La recompensa $r_t = \mathcal{R}(s_t, a_t, s_{t+1})$ tiene dos componentes:

Recompensa inmediata por intento:

$$r_{\text{step}} = \begin{cases} r_{\text{éxito}} + \overline{\Delta \text{ELO}} + r_{\text{topic}} + r_{\text{challenge}} + r_{\text{prog}} + r_{\text{stag}} + r_{\text{overuse}} + r_{\text{div}} & \text{si resuelto} \\ r_{\text{fracaso}} & \text{si fallido} \end{cases}$$

donde los componentes se definen como sigue. Los valores por defecto se indican entre

paréntesis.

Componentes fijos:

- $r_{\text{éxito}} = +10$; $r_{\text{fracaso}} = -2$.
- $\overline{\Delta\text{ELO}}$: media de los incrementos de ELO sobre los temas del problema (ver ec. de actualización de ELO).
- $r_{\text{topic}} = +2$ si el problema cubre al menos un tema canónico nuevo (no visto en la sesión); 0 en caso contrario.

Bonificación por desafío $r_{\text{challenge}}$ (peso $w_c = 4,0$), en función del gap $g = r_p - r_{\text{global}}$:

$$r_{\text{challenge}} = \begin{cases} -4,0 & \text{si } g < -300 \quad (\text{trivial}) \\ -1,5 & \text{si } g > 600 \quad (\text{inalcanzable}) \\ w_c \cdot \frac{g}{200} & \text{si } 0 \leq g \leq 200 \\ w_c \cdot \max\left(0, 1 - \frac{g-200}{400}\right) & \text{si } 200 < g \leq 600 \\ w_c \cdot 0,2 \cdot \left(1 + \frac{g}{300}\right) & \text{si } -300 \leq g < 0 \end{cases}$$

Bonificación por progresión r_{prog} (peso $w_p = 5,0$), en función del incremento de dificultad respecto al problema anterior $\delta = r_p - r_{p-1}$:

$$r_{\text{prog}} = \begin{cases} 0 & \text{si es el primer problema} \\ w_p & \text{si } \delta \geq 300 \\ w_p \cdot \frac{\delta}{300} & \text{si } 0 \leq \delta < 300 \\ \max(-2, \delta/300) & \text{si } \delta < 0 \end{cases}$$

Penalización por estancamiento $r_{\text{stag}} = -6,0$ si el problema actual y los últimos $W - 1$ problemas ($W = 3$) son todos triviales ($r_{\text{global}} - r_p > 300$); 0 en caso contrario.

Penalización por sobreuso de tema r_{overuse} : para cada tema k del problema cuya frecuencia relativa $f_k = n_k/n_{\text{tot}}$ supere el umbral $\tau = 0,30$:

$$r_{\text{overuse}} = \sum_{k: f_k > \tau} (-10) \cdot \frac{f_k - \tau}{1 - \tau}$$

Bonificación por diversidad r_{div} : para cada tema k del problema con $f_k < \tau$ (poco practicado):

$$r_{\text{div}} = \frac{1}{|\text{tags}|} \sum_{k: f_k < \tau} 1,5 \cdot \frac{\tau - f_k}{\tau}$$

Ambas penalizaciones/bonificaciones se activan solo a partir del tercer intento.

Recompensa terminal al finalizar el episodio:

$$r_{\text{terminal}} = 10 \cdot \rho_s(\text{ratings}) + 2 \cdot |\mathcal{C}| + 0,5 \cdot t_{\text{remaining}}$$

donde ρ_s es la correlación de Spearman entre el orden de intento y el rating de los problemas (+1 = perfecto fácil→difícil, -1 = orden inverso), $|\mathcal{C}|$ es el número de temas algorítmicos canónicos distintos cubiertos, y $t_{\text{remaining}}$ es el tiempo restante de sesión en minutos.

Factor de descuento γ . Se usa $\gamma = 0,99$, que valora los refuerzos futuros casi tanto como los inmediatos, incentivando estrategias a largo plazo dentro del episodio.

2.2. Representación de la acción para la red Q

La red Q no recibe directamente el índice de acción, sino el vector concatenado:

$$\mathbf{x}_{t,i} = [s_t \parallel \mathbf{p}_i(s_t)] \in \mathbb{R}^{48}$$

donde $\mathbf{p}_i(s_t) \in \mathbb{R}^{24}$ codifica el problema i en relación con el estado actual del estudiante:

$$\mathbf{p}_i(s_t) = [r_{i,\text{norm}}, \text{gap}_{\text{norm}}, P_{\text{solve}}(p_i \mid s_t), t_{i,\text{norm}}, \text{onehot}(\text{tags}_i)]$$

El agente calcula $Q(s_t, a_i) = f_{\theta}(\mathbf{x}_{t,i})$ para todos los problemas disponibles y selecciona $a_t = \arg \max_i Q(s_t, a_i)$ sobre acciones válidas.

3. Descripción del Dataset

3.1. Fuentes de datos

Se utilizan dos datasets públicos de Codeforces disponibles en Kaggle:

Cuadro 1: Datasets utilizados

Dataset	Fuente	Tamaño	Contenido
<code>codeforces_dataset.csv</code>	Kaggle (immortal3)	14 MB	Contest ID, nombre del problema, enunciado, tags con rating embebido (*XXXX)
<code>codeforces_problemset.csv</code>	Kaggle (erchhh)	20 MB	Enunciado, ejemplos de entrada/salida, límite de tiempo, tags en formato lista Python

Ambos datasets se combinan mediante un proceso de unión por `problem_id` (construido como concatenación de contest ID y letra del problema, e.g. “325A”).

3.2. Pipeline de preprocesamiento

El pipeline transforma los datos crudos en un conjunto de 500 problemas utilizables por el entorno de simulación:

1. **Construcción de `problem_id`:** concatenación de `contest_id` + letra del problema.
2. **Extracción de `rating`:** parseo del token *XXXX embebido en la cadena de tags. Este token es la única fuente de dificultad numérica en ambos datasets.
3. **Limpieza de tags:** eliminación de *XXXX de la lista de tags; normalización a lista de strings en minúsculas.
4. **Filtrado:** descarte de filas sin rating o sin enunciado.
5. **Muestreo estratificado:** selección de 500 problemas distribuidos por bandas de dificultad, con distribución sesgada hacia ratings accesibles para reproducir la distribución real del uso del sistema.

Cuadro 2: Distribución por banda de dificultad en el dataset procesado

Rango de rating	Proporción	Problemas (N=500)
800 – 1199 (Newbie)	25 %	125
1200 – 1599 (Pupil)	30 %	150
1600 – 1999 (Specialist)	25 %	125
2000 – 2399 (Expert)	12 %	60
2400 – 2799 (Candidate Master)	5 %	25
2800+ (Master / Grandmaster)	3 %	15

3.3. Perfiles de estudiante

El entorno requiere estudiantes simulados con distribuciones de habilidades realistas. Se generaron 500 perfiles mediante el LLM local (descrito en la Sección 5), con ratings globales entre 800 y 2000 y distribuciones de ELO por tema correlacionadas según el arquetipo generado. Los perfiles se almacenan en `data/processed/student_profiles.json` para garantizar reproducibilidad de los experimentos.

Los perfiles cubren arquetipos como: especialista en matemáticas débil en grafos, especialista en DP, estudiante principiante, programador competitivo avanzado, etc.

4. Diseño del Algoritmo

4.1. Deep Q-Network (DQN)

Se implementa el algoritmo DQN estándar [1] con las siguientes características:

Arquitectura de la red Q. Un perceptrón multicapa (MLP) con la siguiente topología:

$$\mathbb{R}^{48} \xrightarrow{\text{Linear}} \mathbb{R}^{128} \xrightarrow{\text{ReLU}} \mathbb{R}^{64} \xrightarrow{\text{ReLU}} \mathbb{R}^1$$

La salida es el Q-valor escalar para el par (s_t, a_i) . Se mantienen dos instancias con la misma arquitectura: la *red online* (actualizada en cada paso) y la *red target* (sincronizada periódicamente), lo que estabiliza el entrenamiento al desacoplar los targets de los parámetros entrenados.

Política ε -greedy.

$$a_t = \begin{cases} \text{acción aleatoria válida} & \text{con probabilidad } \varepsilon \\ \arg \max_{i \text{ válido}} Q_{\theta}(s_t, a_i) & \text{en caso contrario} \end{cases}$$

ε decae linealmente de 1.0 a 0.05 durante los primeros 80 % de los episodios de entrenamiento.

Experience replay. Se almacenan transiciones $(s_t, a_t, r_t, s_{t+1}, \text{mask}_{t+1}, \text{done})$ en un buffer circular FIFO de capacidad 10,000. Cada paso de actualización muestrea aleatoriamente un batch de tamaño 64.

Por eficiencia de memoria, la matriz de problemas $\mathbf{P}(s_{t+1}) \in \mathbb{R}^{500 \times 24}$ no se almacena en el buffer (supondría ~ 240 MB a plena capacidad), sino que se reconstruye en el momento de entrenamiento desde `ObservationBuilder`.

Actualización. El objetivo de la ecuación de Bellman con red target es:

$$y_t = r_t + \gamma \cdot \max_{a' \text{ válido}} Q_{\theta^-}(s_{t+1}, a') \cdot (1 - \text{done}_t)$$

La pérdida es MSE: $\mathcal{L} = \mathbb{E}[(y_t - Q_{\theta}(s_t, a_t))^2]$, minimizada con Adam ($\text{lr} = 10^{-3}$). Se aplica gradient clipping con $\|\nabla\| \leq 1,0$ para mayor estabilidad.

4.2. Algoritmos de referencia (baselines)

Se implementaron dos baselines deterministas que comparten la interfaz abstracta `ProblemSelector`:

Greedy. En cada paso selecciona el problema disponible que maximiza la ratio:

$$\text{score}(p) = \frac{P_{\text{solve}}(p) \cdot \Delta \text{ELO}_{\text{esperado}}(p)}{t_{\text{estimado}}(p)}$$

Es un heurístico adaptativo: reacciona a los cambios de ELO y fatiga tras cada intento.

Knapsack 0/1. Al inicio del episodio resuelve el problema de la mochila 0/1 con programación dinámica en $O(N \cdot W)$, donde W es el presupuesto de tiempo discretizado en minutos. El valor de cada problema es:

$$V(p) = r_p + 50 \cdot |\{k \in \text{tags}(p) : k \notin \text{topics_seen}\}|$$

Una vez elegido el conjunto óptimo, los problemas se ejecutan en orden ascendente de dificultad. Es el único baseline *no adaptativo*: toma todas las decisiones al inicio y no reacciona a resultados intermedios.

(El algoritmo Rollout, explorado en experimentos preliminares, mostró un rendimiento muy inferior con recompensas medias negativas y un coste computacional inasumible, por lo que se descartó de la comparativa final.)

5. Rol del LLM en el Sistema

El sistema integra un LLM local (llama3.2:3b vía Ollama) en dos roles funcionalmente distintos y complementarios.

5.1. Rol 1: Generación de perfiles de estudiante realistas

Problema que resuelve. Un perfil de estudiante generado aleatoriamente (con ELOs uniformes e independientes por tema) no es representativo de estudiantes reales, en los que las habilidades están fuertemente correlacionadas: un estudiante con ELO alto en DP tiende a tener ELO razonable en matemáticas, mientras que quien es fuerte en grafos generalmente domina también DFS y árboles.

Mecanismo. Se consulta al LLM con un prompt que solicita un perfil de estudiante de programación competitiva con un arquetipo específico. El LLM responde en formato JSON estructurado con ELOs por tema que reflejan las correlaciones aprendidas de su preentrenamiento sobre datos de la comunidad competitiva:

Listing 1: Estructura del perfil generado por el LLM

```
1 {  
2   "archetype": "Math_specialist_struggling_with_graphs",  
3   "global_rating": 1350,  
4   "session_budget_min": 120,  
5   "topic_ratings": {  
6     "math": 1800, "number_theory": 1700,  
7     "dp": 1200, "graphs": 900, ...  
8   }  
9 }
```

Si Ollama no está disponible, el sistema cae en arquetipos predefinidos (*fallback* determinista), garantizando que los experimentos sean reproducibles independientemente del LLM.

Impacto en el sistema. Perfiles realistas producen condiciones de evaluación más representativas: el agente DQN y los baselines son evaluados sobre una distribución de estudiantes más cercana a la real, lo que da mayor validez externa a la comparación experimental.

5.2. Rol 2: Evaluación pedagógica de sesiones completadas

Problema que resuelve. Las métricas cuantitativas (recompensa total, Spearman, cobertura de temas) no capturan la calidad pedagógica de una secuencia desde una perspectiva didáctica. Dos sesiones con la misma recompensa pueden diferir significativamente en su valor educativo.

Mecanismo. Tras completar una sesión, se construye un prompt que describe al LLM la secuencia completa de problemas intentados (ID, rating, tags, resultado, recompensa) y el estado final del estudiante. El LLM actúa como “coach” experto y devuelve una evaluación estructurada en JSON:

Listing 2: Estructura de la evaluación pedagógica del LLM

```
1 {  
2   "score": 8,  
3   "difficulty_progression": "good",  
4   "topic_diversity": "average",  
5   "challenge_level": "appropriate",  
6   "explanation": "La sesion mostro una progresion ...",  
7   "recommendation": "Incluir mas problemas de grafos ..."  
8 }
```

La puntuación (1–10) y las dimensiones cualitativas (*difficulty progression*, *topic diversity*, *challenge level*) complementan las métricas numéricas con una interpretación en lenguaje natural, accesible para el usuario final.

Por qué el LLM es necesario aquí. La evaluación pedagógica requiere razonamiento sobre conceptos abstractos (“¿esta secuencia es apropiada para este estudiante?”) que no pueden reducirse a una función escalar simple sin perder información semántica relevante. El LLM proporciona esta capacidad de evaluación cualitativa de forma directa, sin necesidad de diseñar y validar una rúbrica manual.

6. Metodología Experimental

6.1. Entrenamiento del agente DQN

El agente se entrenó durante 8,000 episodios sobre el pool de 500 problemas, con perfiles de estudiante rotados secuencialmente del conjunto de 500 perfiles preprocesados (cada perfil se reutiliza 16 veces a lo largo del entrenamiento). Este número de episodios permite que ε alcance su valor mínimo en las primeras 6,400 iteraciones, dejando las últimas 1,600 en régimen casi puramente explotador. Parámetros de entrenamiento:

Cuadro 3: Hiperparámetros de entrenamiento

Parámetro	Valor
Episodios de entrenamiento	8,000
$\varepsilon_{\text{start}}$	1.0
ε_{end}	0.05
Decaimiento de ε	lineal, 80 % de episodios (6,400 ep.)
Learning rate (Adam)	10^{-3}
Batch size	64
Capacidad del buffer	10,000
Sincronización de target	cada 10 episodios
γ	0.99
Gradient clipping	1.0

6.2. Protocolo de evaluación

Se realizaron experimentos con $n = 50$ episodios por algoritmo sobre la misma colección de 50 perfiles de estudiante, garantizando la comparabilidad de los resultados. Los algoritmos evaluados son DQN, Knapsack y Greedy. Las métricas registradas en cada episodio son:

- **Recompensa total:** suma de recompensas inmediatas del episodio.
- **Progresión de Spearman** (ρ_s): correlación de Spearman entre el orden de intento y el rating del problema (+1 = perfecto fácil→difícil).
- **Cobertura temática:** número de temas algorítmicos canónicos distintos cubiertos.
- **Tasa de éxito:** problemas resueltos / intentados.

- **Tiempo usado minutos:** tiempo total consumido en la sesión.
- **Mejora de rating:** ELO global final – ELO global inicial.

La significancia estadística se evalúa con el test de Mann-Whitney U (no paramétrico, apropiado dada la distribución no normal de las métricas) con nivel de significancia $\alpha = 0,05$.

7. Resultados y Análisis

Los experimentos se realizaron con 50 perfiles de estudiante diferentes (50 episodios) para cada uno de los tres selectores principales: DQN, Knapsack y Greedy. El algoritmo Rollout se descartó en esta fase final por su bajo rendimiento en experimentos preliminares. Las Tablas 4 y ?? resumen las métricas obtenidas.

Cuadro 4: Resultados agregados: DQN, Knapsack y Greedy (50 episodios cada uno)

Selector	Recompensa <i>media (sd)</i>	Spearman <i>media (sd)</i>	Cobertura <i>media (sd)</i>	Tasa éxito <i>media (sd)</i>	Tiempo usado <i>media (sd)</i>	Mejora rating <i>media (sd)</i>
DQN	38.17 (21.99)	0.422 (0.404)	4.16 (1.98)	0.568 (0.213)	119.92 (28.05)	48.99 (133.58)
Knapsack	31.89 (21.04)	0.602 (0.324)	3.10 (1.58)	0.460 (0.188)	119.39 (28.34)	56.40 (134.80)
Greedy	16.60 (19.94)	0.065 (0.438)	6.30 (1.74)	0.435 (0.177)	118.95 (28.27)	35.59 (122.30)

7.1. Análisis de la recompensa total

El DQN obtiene la recompensa media más alta (38.17), superando al Knapsack (31.89) en 6.28 puntos y al Greedy (16.60) en más del doble. La diferencia entre DQN y Greedy es estadísticamente significativa (Mann-Whitney U, $p < 0,0001$). Aunque la diferencia entre DQN y Knapsack no alcanza el umbral de significancia convencional ($p = 0,172$), la ventaja numérica del DQN es consistente a lo largo de los 50 episodios.

7.2. Progresión de dificultad (Spearman)

El Knapsack mantiene la mejor progresión de dificultad (0.602), muy superior a la del DQN (0.422) y a la prácticamente nula del Greedy (0.065). Las diferencias son significativas: DQN vs. Knapsack ($p = 0,008$) y DQN vs. Greedy ($p < 0,0001$). El orden ascendente de dificultad del Knapsack es estructuralmente óptimo para esta métrica, pero no se traduce en una mejor recompensa total.

7.3. Cobertura temática

El Greedy cubre la mayor cantidad de temas distintos (6.30), seguido por el DQN (4.16) y finalmente el Knapsack (3.10). Sin embargo, la alta cobertura del Greedy viene acompañada de una progresión de dificultad deficiente y una recompensa total baja. El DQN equilibra cobertura y progresión, mientras que el Knapsack sacrifica cobertura para maximizar la progresión. Todas las diferencias son significativas ($p < 0,01$).

7.4. Tasa de éxito

El DQN presenta la mayor tasa de éxito (0.568), superando al Knapsack (0.460) y al Greedy (0.435). La diferencia entre DQN y Knapsack es significativa ($p = 0,009$) y también entre DQN y Greedy ($p = 0,001$). Esto indica que el DQN aprende a seleccionar problemas que el estudiante virtual tiene una probabilidad realista de resolver, sin caer en la sobreexplotación de problemas triviales (como sucedía con el Rollout en experimentos previos).

7.5. Mejora de rating

El Knapsack obtiene la mayor mejora media de rating (56.40), seguido del DQN (48.99) y el Greedy (35.59). Las diferencias no son estadísticamente significativas debido a la alta varianza de esta métrica (desviación típica ~ 130 puntos). La mejora de rating está fuertemente influenciada por el perfil inicial del estudiante y por la aleatoriedad del modelo de éxito/fracaso.

7.6. Análisis cualitativo por tipos de perfil

El DQN destaca especialmente en perfiles de estudiante asimétricos o con puntos débiles muy marcados. Por ejemplo, en el arquetipo “Math specialist struggling with graphs” (episodio 2), el DQN alcanza una recompensa de 62.28 frente a 0.32 del Knapsack y 24.65 del Greedy. En “Advanced Data Structures and Algorithms Specialist” (episodio 30), el DQN obtiene 82.87, superando ampliamente al Knapsack (50.17) y al Greedy (37.85).

Por el contrario, en perfiles muy homogéneos o con todos los ELOs equilibrados, el Knapsack puede ser competitivo o ligeramente superior, pero la ventaja del DQN en escenarios realistas (habilidades no uniformes) respalda su elección como algoritmo principal para el sistema.

7.7. Síntesis de los resultados

Los experimentos con 50 episodios por algoritmo confirman que el DQN aprende una política que maximiza la recompensa total, combinando una tasa de éxito alta con una cobertura temática intermedia y una progresión de dificultad aceptable. El Knapsack es el baseline más competitivo, especialmente en progresión de dificultad, pero su naturaleza estática lo limita ante perfiles de estudiante heterogéneos. El Greedy, aunque cubre muchos temas, carece de progresión y obtiene la peor recompensa. Estos resultados validan el enfoque de aprendizaje por refuerzo profundo para el problema de selección óptima de problemas bajo restricciones de tiempo.

8. Limitaciones y Posibles Mejoras

8.1. Limitaciones actuales

Modelo del estudiante simplificado. El `StudentModel` es una aproximación probabilística, no un modelo validado empíricamente. Las constantes de fatiga, actualización de ELO y tiempo de resolución se calibraron manualmente. Un modelo entrenado con datos reales de Codeforces (tiempos de envío, historial de aceptaciones) produciría simulaciones más fieles.

Ausencia de Double DQN y Prioritized Replay. La implementación usa DQN estándar. Extensiones conocidas como Double DQN (reduce sobreestimación de Q-valores) y Prioritized Experience Replay (muestra transiciones más informativas) podrían mejorar la convergencia y la calidad de la política aprendida.

Pool de problemas fijo. El pool de 500 problemas es estático. En un sistema real, el pool debería actualizarse periódicamente con nuevos problemas de Codeforces. La generalización del agente a problemas no vistos durante el entrenamiento no ha sido evaluada.

Evaluación LLM sin validación humana. Las puntuaciones pedagógicas del LLM no han sido comparadas con evaluaciones de instructores humanos. La validez del LLM como evaluador pedagógico es una hipótesis razonable pero no verificada en este trabajo.

Varianza alta en rating improvement. La desviación estándar de la mejora de rating (~ 110 – 130 puntos) es muy elevada en todos los algoritmos, lo que indica que esta métrica está fuertemente influenciada por el perfil inicial del estudiante y la aleatoriedad del modelo. Se necesitan más episodios o estratificación por rating inicial para obtener estimaciones más precisas.

8.2. Posibles mejoras

1. **Double DQN / Dueling DQN:** incorporar estas variantes para reducir la sobreestimación de Q-valores y mejorar la estabilidad del entrenamiento.
2. **Modelo del estudiante basado en datos reales:** entrenar los parámetros del modelo con historiales de resolución reales de Codeforces.
3. **LLM como función de recompensa aumentada:** en lugar de usar el LLM solo como evaluador *post-hoc*, integrar su puntuación pedagógica directamente en la función de recompensa durante el entrenamiento (reward shaping).
4. **Pool dinámico de problemas:** conectar el sistema a la API de Codeforces para actualizar el pool con problemas recientes.
5. **Generalización a otros jueces en línea:** extender el preprocesador para soportar datasets de LeetCode o AtCoder.
6. **Validación con usuarios reales:** comparar las sesiones recomendadas por el DQN con sesiones diseñadas por instructores expertos.

Referencias

- [1] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [3] A. E. Elo, *The Rating of Chessplayers, Past and Present*. Arco Publishing, 1978.
- [4] M. Towers et al., “Gymnasium,” *arXiv:2407.17032*, 2024.
- [5] Ollama, “Run large language models locally,” <https://ollama.com>, 2024.
- [6] Codeforces, “Competitive programming judge,” <https://codeforces.com>.